

Detection pipeline (Marker Detection)

PHỤ THUỘC A/01-mo-hình-camera-va-distortion (mục 4 — vì sao méo chưa khử phá hoại chính bước khớp cạnh mà chương này mô tả); A/02-homography-va-hình-hoc-phang (homography để rectify vùng tag trước khi đọc bit).

TÓM TẮT

Chương này mô tả quy trình biến một ảnh xám thành một danh sách tag kèm tọa độ góc, và mục đích của nó không phải để bạn tự cài lại — các cài đặt sẵn có đều tốt — mà để bạn biết *sai số sinh ra ở bước nào*. Ba bước đáng chú ý: ngưỡng hoá quyết định tag có được phát hiện không, khớp cạnh quyết định σ , và giải mã quyết định tỉ lệ báo sai. Điều đáng nói thẳng ngay: *đây là chương có đòn bẩy thấp nhất trong Phần B*. Chọn detector nào ảnh hưởng tới σ chừng hai tới ba lần, trong khi bố trí constellation ảnh hưởng tới sai số góc hàng chục lần (B/09). Nếu chỉ nhớ một điều: *AprilTag xuất sắc ở việc nhận diện và bắt bám, không xuất sắc ở việc định vị góc* — và sự phân biệt ấy dẫn thẳng tới đề xuất dùng target lại ở B/08.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ngưỡng hoá thích nghi (adaptive threshold)** — biến ảnh xám thành ảnh nhị phân với ngưỡng tính *cục bộ* theo từng vùng thay vì một ngưỡng chung. Cần thiết vì chiếu sáng không đều: một ngưỡng chung làm mất tag ở vùng tối hoặc vùng sáng.
- **thành phần liên thông (connected component)** — tập các pixel cùng giá trị nhị phân và liền kề nhau. Tìm chúng hiệu quả bằng cấu trúc *union-find*, và chúng là ứng viên cho các vùng tag.
- **khớp tứ giác (quad fitting)** — tìm bốn cạnh và bốn góc từ một thành phần liên thông có hình dạng gần tứ giác. Đây là bước quyết định σ .
- **khớp đường có trọng số gradient** — thay vì dùng các pixel biên của ảnh nhị phân, dùng các pixel có gradient lớn trong ảnh *xám gốc*, với trọng số theo độ lớn gradient. Đây là bước làm AprilTag 2 chính xác hơn hẳn phiên bản đầu [1].
- **rectify** — dùng homography đưa vùng tứ giác trên ảnh về một hình vuông chuẩn, để việc lấy mẫu các ô bit trở thành việc đọc một lưới đều A/02.
- **khoảng cách Hamming** — số vị trí bit khác nhau giữa hai từ mã. Một từ điển có khoảng cách tối thiểu d thì sửa được $\lfloor (d-1)/2 \rfloor$ lỗi bit và phát hiện được $d-1$ lỗi.
- **tỉ lệ báo sai (false positive rate)** — xác suất một vùng ảnh ngẫu nhiên được giải mã thành một ID hợp lệ. Nó là đại lượng an toàn quan trọng nhất của một hệ thống marker, và nó đánh đổi trực tiếp với khả năng phát hiện ở khoảng cách xa.

CÂU HỎI MỞ ĐẦU

- Q1. Detector của bạn báo phát hiện được tag ID 7 ở một vị trí mà bạn biết chắc không có tag nào. Nêu hai cơ chế hoàn toàn khác nhau có thể gây ra điều đó, và nêu hai biện pháp phòng ở hai tầng khác nhau.
- Q2. AprilTag 2 chính xác hơn AprilTag 1 đáng kể, và cải tiến chính nằm ở đúng một bước của pipeline. Bước nào, và vì sao thay đổi ấy lại quan trọng đến thế?
- Q3. ChArUco dùng ArUco để định danh nhưng *không* dùng góc của ArUco để đo. Vì sao lại chia việc như vậy, và điều đó nói gì về điểm mạnh riêng của mỗi loại?

- Q4. Bạn chọn họ tag có nhiều ô bit hơn để có nhiều ID hơn và khoảng cách Hamming lớn hơn. Nếu cái giá phải trả, và tính xem nó ảnh hưởng tới khoảng cách phát hiện tối đa như thế nào.
- Q5. Trong tám chương của Phần B, chương này có đòn bẩy thấp nhất đối với ngân sách sai số. Vì sao điều đó đúng, và vì sao nó vẫn đáng đọc?

1.1 Đặt vấn đề

Mọi thứ trong cây bắt đầu từ một danh sách: tag nào nhìn thấy, và bốn góc của mỗi tag nằm ở đâu trên ảnh. Chương này mô tả cách danh sách ấy được tạo ra.

Nhưng mục đích của chương không phải để bạn tự cài lại một detector. Các cài đặt có sẵn — `apriltag`, `OpenCV objdetect` — đều tốt, được kiểm nghiệm rộng rãi, và tối ưu hơn nhiều so với thứ bạn viết trong một tuần. Mục đích là để bạn biết *sai số sinh ra ở bước nào*, vì ba lý do thực dụng.

Lý do thứ nhất: các bước khác nhau hỏng theo những cách khác nhau, và triệu chứng khác nhau. Ngưỡng hoá hỏng thì tag *biến mất* hoàn toàn. Khớp cạnh hỏng thì tag vẫn được phát hiện nhưng σ lớn. Giải mã hỏng thì bạn có báo sai hoặc bỏ sót. Ba triệu chứng ấy đòi ba cách chữa hoàn toàn khác nhau, và nhầm lẫn giữa chúng làm mất nhiều thời gian.

Lý do thứ hai: có một vài tham số trong các cài đặt sẵn có ảnh hưởng thật, và biết pipeline là biết chỉnh cái nào. Ngưỡng kích thước tối thiểu, có bật tinh chỉnh cạnh hay không, số lỗi bit cho phép sửa — ba tham số này đáng chỉnh và phần còn lại thường nên để mặc định.

Lý do thứ ba, và nó là lý do quan trọng nhất cho bài toán 5 mm: hiểu pipeline cho biết *giới hạn* của từng loại marker. Cụ thể, nó giải thích vì sao góc AprilTag đến từ giao hai đường thẳng còn góc chessboard đến từ một điểm yên ngựa, và vì sao hai cơ chế ấy cho độ chính xác khác nhau. Kết luận ấy dẫn thẳng tới đề xuất target lai ở B/08, và nó là một trong ba luận điểm “nói ngược thói quen” ở 00-map.

Cần nói thẳng một điều ngay từ đầu, vì nó ảnh hưởng tới việc bạn nên đầu tư bao nhiêu thời gian vào chương này: *đây là chương có đòn bẩy thấp nhất trong Phần B*. Chuyển từ detector này sang detector khác thay đổi σ chừng hai tới ba lần. Chuyển từ một tag đơn sang constellation có baseline thay đổi sai số góc hàng chục lần (A/05 mục 6). Với một dự án có thời gian hạn chế, thứ tự đầu tư đúng là B/13, B/09, B/12, rồi mới tới đây. Tôi đặt chương này ở đầu Phần B vì nó là chỗ chuỗi đo bắt đầu, không phải vì nó quan trọng nhất — và sự khác biệt ấy đáng được nói ra thay vì để người đọc tự suy.

TRỰC GIÁC

Hình dung việc tìm một cái biển báo trong một bức ảnh chụp phố.

Việc đầu tiên là tách sáng khỏi tối. Nhưng không thể dùng một ngưỡng chung cho cả ảnh, vì một góc phố có nắng và một góc có bóng râm — cái ngưỡng đúng cho vùng này thì sai cho vùng kia. Nên phải hỏi từng vùng nhỏ một: ở *quanh đây*, cái gì là sáng và cái gì là tối? Việc thứ hai là gom các pixel tối liên nhau thành từng mảng, rồi hỏi mảng nào trông giống một hình tứ giác. Phần lớn bị loại ngay — quá nhỏ, quá méo, quá nhiều lỗ.

Việc thứ ba là chỗ quan trọng nhất, và cũng là chỗ tinh tế nhất. Ta đã có một mảng pixel hình tứ giác, nhưng cái ta cần là *toạ độ bốn góc, chính xác tới phần mười pixel*. Cách ngây thơ là lấy bốn pixel ngoài cùng của mảng — và cách đó tệ, vì nó chỉ dùng bốn pixel và vứt bỏ toàn bộ thông tin trong hàng trăm pixel dọc các cạnh.

Cách tốt là: nhìn vào *ảnh xám gốc* chứ không phải ảnh đen trắng, tìm các pixel nơi độ sáng đổi nhanh nhất, kẻ một đường thẳng qua chúng theo cách tốt nhất, rồi lấy chỗ hai đường cắt nhau. Bây giờ mỗi góc được quyết định bởi hàng trăm pixel thay vì một pixel — và đó là toàn bộ lý do độ chính xác dưới pixel là chuyện có thể.

Việc cuối là đọc mã. Nấn cái tứ giác trở lại thành hình vuông, chia thành lưới ô, hỏi từng ô sáng hay tối, rồi tra xem chuỗi bit ấy có trong danh sách hợp lệ không. Bước tra ấy quan trọng hơn vẻ ngoài: nó là thứ ngăn một vết bẩn ngẫu nhiên trên tường được báo cáo là một cái tag.

1.2 AprilTag: bốn bước và chỗ σ sinh ra

Mô tả theo phiên bản 2 và 3 [1]–[3], vì đó là những phiên bản đang dùng.

Bước một, ngưỡng hoá thích nghi. Chia ảnh thành các ô nhỏ, tính cực đại và cực tiểu độ sáng trong mỗi ô cùng các ô lân cận, và đặt ngưỡng ở giữa. Các ô có độ tương phản thấp bị đánh dấu là “không xác định” và bỏ qua — một chi tiết quan trọng vì nó ngăn việc khuếch đại nhiễu trong vùng đồng nhất.

Bước hai, phân đoạn thành thành phần liên thông. Dùng union-find để gom các pixel cùng giá trị thành từng nhóm, rồi tìm các *biên* giữa vùng sáng và vùng tối. Các biên này là ứng viên cho cạnh tag.

Bước ba, khớp tứ giác. Nhóm các đoạn biên thành các chuỗi khép kín, kiểm xem chuỗi có thể xấp xỉ bằng bốn đoạn thẳng không, và loại các ứng viên quá nhỏ hoặc quá méo. Đây là chỗ tham số `min_tag_size` hoặc tương đương có tác dụng, và nó là tham số đáng chỉnh theo A/02 mục 4.

Bước bốn, và đây là bước quyết định σ : khớp đường có trọng số gradient. Thay vì dùng các pixel biên của ảnh nhị phân — vốn chỉ chính xác tới một pixel — AprilTag 2 quay lại ảnh xám gốc, lấy các pixel dọc mỗi cạnh, và khớp một đường thẳng với trọng số theo độ lớn gradient tại từng pixel [1]. Bốn góc là giao của bốn đường thẳng ấy.

Đây là câu trả lời cho Q2, và lý do cải tiến này quan trọng đến thế đáng nói rõ. Với ảnh nhị phân, thông tin về vị trí cạnh bị lượng tử hoá thành một pixel và mọi thông tin về *độ sáng* bị vứt bỏ. Với ảnh xám và trọng số gradient, một cạnh đi qua giữa hai pixel để lại dấu vết ở tỉ lệ độ sáng của hai pixel ấy, và thông tin đó khôi phục được vị trí cạnh tới phần nhỏ của một pixel. Cộng thêm việc dùng cả trăm pixel dọc cạnh để xác định một đường, và độ chính xác cải thiện đáng kể.

Sau đó là giải mã: rectify bằng homography (A/02), lấy mẫu các ô bit, so với từ điển với sửa lỗi Hamming, và xác định hướng (một hình vuông có bốn hướng khả dĩ, và mã phải phân biệt được chúng — các họ tag được thiết kế sao cho bốn phép xoay của một từ mã không trùng với từ mã nào khác).

CẦN NHỚ

Bốn góc AprilTag đến từ *giao của bốn đường thẳng*. Ba hệ quả quan trọng của sự thật ấy, và cả ba đều xuất hiện lại ở các chương khác.

Một, độ chính xác phụ thuộc vào *chiều dài cạnh* chứ không chỉ vào kích thước tag — một cạnh dài cho nhiều pixel gradient hơn nên đường khớp tốt hơn.

Hai, sai số ở giao điểm bị *khuếch đại* khi hai đường gần song song, tức khi tag nhìn rất chếch. Đây là một nguồn làm Σ_i khác nhau giữa các góc (A/03 mục 6).

Ba, và đây là hệ quả nghiêm trọng nhất: *giả định “cạnh là đường thẳng” phải đúng*. Méo chưa khử làm cạnh cong, và khớp thẳng lên cong cho đường sai (A/01 mục 4). Đây là lý do chương 1 phải đọc trước chương này.

1.3 ArUco và ChArUco

A. ArUco

Quy trình khác ở chi tiết nhưng cùng cấu trúc: ngưỡng hoá thích nghi, tìm contour, xấp xỉ contour bằng đa giác và giữ các đa giác bốn cạnh, rectify bằng homography, lấy mẫu bit, tra từ điển [4].

Đóng góp riêng của ArUco không nằm ở phần phát hiện mà ở phần *từ điển*: các tác giả đề xuất sinh từ điển bằng cách tối ưu hoá khoảng cách Hamming giữa các từ mã và giữa các phép xoay của chúng, cho phép tạo từ điển với kích thước và mức bền tùy chọn [5]. Đây là một lợi thế thực dụng: bạn chọn được từ điển nhỏ với khoảng cách lớn, đúng thứ cần cho docking nơi số ID cần thiết rất ít.

Về độ chính xác định vị góc, cài đặt cơ bản của ArUco dùng góc của đa giác xấp xỉ, kém hơn cơ chế khớp đường có trọng số của AprilTag. OpenCV có tùy chọn tinh chỉnh góc bằng cornerSubPix sau đó, và *nên bật nó* — đây là một trong vài tham số đáng chỉnh.

B. ChArUco: chia việc theo điểm mạnh

Đây là thiết kế đáng chú ý nhất trong mục này và nó trả lời Q3.

ChArUco là một bàn cờ vua với các ô ArUco đặt trong các ô trắng. Quy trình: phát hiện các marker ArUco để biết *đang nhìn phần nào của bàn cờ* (định danh), rồi dự đoán vị trí các góc chessboard và tinh chỉnh chúng bằng thuật toán điểm yên ngựa (đo).

Sự chia việc ấy khai thác đúng điểm mạnh của từng loại. ArUco *giải định danh*: nó cho biết ô nào là ô nào, kể cả khi chỉ nhìn thấy một phần bàn cờ — điều mà một bàn cờ thuần không làm được vì mọi góc trông giống nhau. Chessboard *giải đo*: góc của nó là một điểm yên ngựa với cấu trúc đối xứng hai chiều, và định vị một điểm như thế chính xác hơn định vị giao của hai đường thẳng.

Vì sao chính xác hơn? B/08 bàn kỹ, nhưng bản chất là: một điểm yên ngựa có gradient đổi dấu theo *cả hai* hướng quanh nó, nên vị trí của nó bị ràng buộc từ bốn phía; một giao điểm chỉ bị ràng buộc bởi hai đường, và sai số của mỗi đường lan vào giao điểm với hệ số khuếch đại phụ thuộc góc giao.

Đây chính là cơ sở cho đề xuất *target lai* ở 00-map: dùng AprilTag để bắt bám và định danh từ xa — nơi nó xuất sắc — và một vùng chessboard hoặc ChArUco cho phép đo tinh ở cự ly gần — nơi độ chính xác góc quyết định.

1.4 Các họ marker khác

Ngắn gọn, vì với bài toán docking thì chúng phần lớn là tham khảo.

Deltile [6]: lưới tam giác thay vì lưới vuông. Lập luận: một đỉnh tam giác được bao quanh bởi sáu vùng thay vì bốn, nên có nhiều thông tin hơn để định vị, và các tác giả trình bày một phép tinh chỉnh saddle point bậc hai riêng cho nó. Đáng biết khi cần hiệu chuẩn intrinsic chất lượng cao nhất.

Marker tròn đồng tâm: WhyCon [7] và CCTag [8]. Định vị tâm rất chính xác vì tâm của một vùng lớn được xác định bởi rất nhiều pixel biên. Nhưng — và đây là cảnh báo phải lặp lại — chúng chịu bias tâm ellipse (A/σ^2 mục 6), một thiên lệch hệ thống phụ thuộc tư thế. Với bài toán 5 mm, một phép đo rất chính xác quanh một giá trị thiên lệch không bằng một phép đo kém chính xác hơn mà không thiên lệch, trừ khi bạn hiệu chỉnh conic đúng cách.

STag [9]: thêm một đường tròn ngoại tiếp vào thiết kế tag để ổn định pose. Triết lý giống chương này: giải quyết ở tầng thiết kế marker thay vì tầng thuật toán.

Detector học sâu: DeepTag [10], DeepArUco [11]. Bền hơn với mờ, che khuất, chiếu sáng khắc nghiệt. Nhưng *độ chính xác góc chưa chắc hơn* — đây là một nhận định quan trọng và tôi phát biểu nó một cách thận trọng: các công trình ấy tập trung vào tỉ lệ phát hiện trong điều kiện khó, và độ chính xác định vị góc dưới điều kiện tốt không phải trọng tâm của chúng. (*Tôi không có dữ liệu so sánh được để khẳng định chiều nào tốt hơn; tôi chỉ nói rằng không nên giả định rằng bền hơn thì cũng chính xác hơn.*) Với docking — nơi điều kiện kiểm soát được và độ chính xác là mục tiêu — lợi ích của chúng ít rõ ràng hơn so với các ứng dụng ngoài trời.

Bảng 1.1. Các họ marker: cơ chế định vị góc và vai trò trong bài toán docking.

Họ	Cơ chế định vị	Vai trò cho docking
AprilTag	Giao của bốn đường khớp có trọng số gradient [1]	Nhận diện và bắt bám từ xa; tỉ lệ báo sai rất thấp
ArUco	Góc đa giác, nên bật tính chỉnh cornerSubPix	Tương tự AprilTag; ưu thế ở từ điển tùy chỉnh được [5]
ChArUco	ArUco định danh, saddle point chessboard để đo	Đo tinh ở cự ly gần; nửa còn lại của target lai
Deltile [6]	Saddle point trên lưới tam giác	Hiệu chuẩn intrinsic chất lượng cao ($B/12$)
Tròn đồng tâm	Tâm ellipse, cần hiệu chỉnh conic	σ rất nhỏ nhưng có thiên lệch phụ thuộc tư thế — cần trọng
Học sâu	Học từ dữ liệu	Bền với điều kiện khó; độ chính xác góc chưa rõ hơn

1.5 Mã hoá, khoảng cách Hamming, và tỉ lệ báo sai

Mục này trả lời Q1 và Q4, và nó là mục về *an toàn* chứ không về độ chính xác.

A. Vì sao báo sai nguy hiểm hơn bỏ sót

Trong hệ docking, hai loại lỗi có hậu quả rất khác nhau. *Bỏ sót* — không phát hiện được tag đang có — làm robot chờ hoặc retry; phiền nhưng an toàn. *Báo sai* — phát hiện một tag không tồn tại, hoặc giải mã sai ID — làm robot lái về một chỗ sai với sự tự tin đầy đủ. Hậu quả có thể là va chạm.

Đây là lý do mọi họ tag hiện đại đầu tư nặng vào mã hoá, và tại sao AprilTag được thiết kế với tỉ lệ báo sai rất thấp làm mục tiêu thiết kế chính [2].

B. Cơ chế mã hoá

Từ điển tag là một tập từ mã nhị phân được chọn sao cho khoảng cách Hamming tối thiểu giữa hai từ mã bất kỳ — và giữa mọi phép xoay của chúng — lớn nhất có thể. Tên họ tag mã hoá thông tin này: 36h11 nghĩa là 36 bit dữ liệu (lưới 6×6) với khoảng cách Hamming tối thiểu 11.

Từ lý thuyết mã: với khoảng cách tối thiểu d , sửa được $t = \lfloor (d - 1)/2 \rfloor$ lỗi bit và phát hiện được tới $d - 1$ lỗi. Với 36h11: sửa được 5 bit, phát hiện được 10.

Tham số thực hành quan trọng nhất ở đây là *số lỗi bit cho phép sửa* — các thư viện cho phép đặt nó. Đặt cao thì phát hiện được tag bẩn hoặc mờ, đổi lại tỉ lệ báo sai tăng. Với docking, nơi bạn biết trước có đúng những ID nào, khuyến nghị là: *đặt số lỗi cho phép ở mức thấp và bù lại bằng whitelist ID*. Whitelist gần như miễn phí và nó giảm tỉ lệ báo sai theo tỉ lệ giữa số ID bạn dùng và kích thước từ điển.

C. Đánh đổi số ô bit và khoảng cách phát hiện

Đây là phần thứ hai của Q4, và nó là một phép tính đáng làm.

Một tag có $n \times n$ ô dữ liệu cộng viền, tổng khoảng $(n + 2) \times (n + 2)$ ô trên cạnh. Để giải mã tin cậy cần chừng 3–5 px mỗi ô (A/06 mục 4), nên kích thước ảnh tối thiểu là $w_{\min} \approx 4(n + 2)$ px.

So sánh hai họ: 16h5 (4×4 ô dữ liệu, tổng 6×6) cần $w_{\min} \approx 24$ px; 36h11 (6×6 , tổng 8×8) cần $w_{\min} \approx 32$ px.

Vì $w = fW/Z$, khoảng cách phát hiện tối đa tỉ lệ nghịch với $w_{\min}$:

$$\frac{Z_{\max}(16h5)}{Z_{\max}(36h11)} = \frac{32}{24} = 1,33.$$

Tức họ ít ô hơn cho tầm phát hiện xa hơn 33% với cùng kích thước tag vật lý — hoặc, nhìn ngược lại, cho phép dùng tag nhỏ hơn 33% cho cùng tầm.

Với docking, số ID cần thiết thường dưới vài chục, trong khi 36h11 có hàng trăm. Nên đánh đổi gần như luôn nghiêng về phía *ít ô hơn*, và mất mát về khoảng cách Hamming được bù bằng whitelist. Đây là một chỗ mà lựa chọn mặc định — dùng 36h11 vì nó là mặc định của thư viện — không phải lựa chọn tốt nhất, và biết pipeline là biết điều đó.

CẦN NHỚ

Ba tham số đáng chỉnh trong một detector, và phần còn lại nên để mặc định.

Kích thước tag tối thiểu: đặt theo ngưỡng đo được của A/02 mục 4, không theo mặc định. Từ chối phát hiện dưới ngưỡng thay vì trả về một pose kém.

Tính chỉnh góc: bật. Với ArUco đây là tùy chọn và nó tắt mặc định ở một số phiên bản. Với AprilTag thì cơ chế khớp đường đã có sẵn.

Số lỗi bit cho phép sửa: đặt thấp, và bù bằng whitelist ID. An toàn quan trọng hơn tầm phát hiện thêm vài chục phân.

Và một quyết định đứng trên cả ba: *chọn họ tag ít ô bit nhất đủ dùng*. Docking cần vài chục ID, không cần hàng trăm.

1.6 Giới hạn và chế độ hồng

Ngưỡng hoá hồng khi tương phản thấp hoặc không đều. Triệu chứng: tag biến mất hoàn toàn, thường ở một số vùng ảnh hoặc một số điều kiện chiếu sáng. Chữa ở tăng chiếu sáng (B/14), không ở tăng thuật toán.

Khớp tứ giác nhầm với vật thể khác. Cửa sổ, biển báo, hoạ tiết sàn — bất cứ thứ gì gần tứ giác — đều là ứng viên. Đây là một trong hai cơ chế của Q1, và nó được lọc ở bước giải mã.

Giải mã sai do nhiễu bit. Cơ chế thứ hai của Q1. Với số lỗi cho phép cao và từ điển lớn, xác suất một vùng ngẫu nhiên giải mã thành ID hợp lệ không phải là không đáng kể. Chữa: giảm số lỗi cho phép, whitelist ID, và — ở tầng cao hơn — kiểm tính hợp lý về pose (một tag báo ở khoảng cách 50 m hoặc ở góc nghiêng 89° thì đáng ngờ).

Sai hướng khi giải mã. Hình vuông có bốn hướng, và các họ tag được thiết kế để phân biệt chúng — nhưng một tag rất mờ hoặc rất nhỏ có thể giải mã đúng ID mà sai hướng. Hậu quả: pose sai đúng 90° với sai số tái chiếu vẫn nhỏ (A/03 mục 8). Bắt bằng kiểm nhất quán liên tag (B/09 mục 7).

Chương này không so sánh định lượng các detector. Đã nói ở mục 4: tôi không có dữ liệu so sánh được. Có công trình so sánh thực nghiệm nhiều họ tag [12], và nó là nơi đáng đọc nếu cần con số — nhưng con số ở đó gắn với điều kiện đo của họ, và việc chuyển sang cấu hình của bạn thì không hiển nhiên. Cách đúng là đo σ trên chính hệ của mình (C/16).

1.7 Thực hành

Bốn việc.

Một, đo σ của chính hệ bạn thay vì tra bảng. Cách đo: cố định camera và tag, chụp vài trăm khung, tính độ lệch chuẩn của toạ độ từng góc. Con số ấy là đầu vào cho mọi phép tính ở C/15, và nó là con số duy nhất trong chuỗi mà chương này ảnh hưởng tới.

Hai, chỉnh ba tham số ở mục 5C và để phần còn lại mặc định.

Ba, bật whitelist ID và ghi log mỗi lần một ID ngoài whitelist được phát hiện — tần suất ấy là thước đo trực tiếp của tỉ lệ báo sai trong môi trường thật của bạn.

Bốn, cân nhắc target lai nếu ngân sách góc chặt: AprilTag để bắt bám, vùng ChArUco để đo tinh. B/08 bàn đầy đủ.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Bốn bước của pipeline AprilTag	Ngưỡng hoá thích nghi; phân đoạn thành phần liên thông (union-find); khớp tứ giác; khớp đường có trọng số gradient rồi lấy giao [1]. Bước bốn quyết định σ .
••• Cải tiến chính của AprilTag 2	Quay lại <i>ảnh xám gốc</i> thay vì ảnh nhị phân, khớp đường với trọng số theo độ lớn gradient. Ảnh nhị phân lượng tử vị trí cạnh thành một pixel và vứt bỏ thông tin độ sáng; ảnh xám giữ lại nó nên khôi phục được vị trí dưới pixel.
••• Góc AprilTag đến từ đâu và ba hệ quả	Từ <i>giao của bốn đường thẳng</i> . Hệ quả: độ chính xác phụ thuộc chiều dài cạnh; sai số bị khuếch đại khi hai đường gần song song (tag chéo); và giả định “cạnh thẳng” phải đúng nên méo chưa khử là chí mạng (A/01 mục 4).
••• ChArUco chia việc thế nào và vì sao	ArUco <i>định danh</i> (biết đang nhìn phần nào của bàn cờ, kể cả khi chỉ thấy một phần); góc chessboard để <i>đo</i> (saddle point có gradient đổi dấu theo cả hai hướng nên bị ràng buộc từ bốn phía, chính xác hơn giao hai đường).
••• Cơ sở của target lai	AprilTag xuất sắc ở nhận diện và bắt bám từ xa; chessboard xuất sắc ở định vị góc. Dùng cả hai theo đúng điểm mạnh — đây là một trong ba luận điểm “nói ngược thói quen” ở 00-map.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Vì sao báo sai nguy hiểm hơn bỏ sót	Bỏ sót $\Rightarrow$ robot chờ hoặc retry, an toàn. Báo sai $\Rightarrow$ robot lái về chỗ sai với sự tự tin đầy đủ, có thể va chạm. Đây là lý do mã hoá được đầu tư nặng [2].
●● Tên họ tag mã hoá gì	$36h11 = 36$ bit dữ liệu (lưới 6×6) với khoảng cách Hamming tối thiểu 11. Sửa được $\lfloor (d-1)/2 \rfloor = 5$ bit, phát hiện được $d-1 = 10$.
●●● Đánh đổi số ô bit	$w_{\min} \approx 4(n+2)$ px. 16h5 cần 24 px, 36h11 cần 32 px $\Rightarrow$ họ ít ô cho tầm phát hiện xa hơn 33%. Docking chỉ cần vài chục ID nên đánh đổi nghiêng về ít ô, bù bằng whitelist.
●●● Ba tham số đáng chỉnh	Kích thước tag tối thiểu (theo ngưỡng đo được, không theo mặc định); bật tinh chỉnh góc (ArUco tắt mặc định ở một số phiên bản); số lỗi bit cho phép <i>thấp</i> , bù bằng whitelist ID.
●● Hai cơ chế gây báo sai	Khớp tứ giác nhầm vật thể khác (cửa sổ, biển báo, hoạ tiết sàn); và giải mã sai do nhiễu bit với số lỗi cho phép cao. Hai biện pháp ở hai tầng: giảm số lỗi cho phép, và whitelist ID cộng kiểm hợp lý pose.
●● Chế độ hóng thâm lặng nhất của chương	Giải mã đúng ID nhưng <i>sai hướng</i> : pose lệch đúng 90° mà sai số tái chiếu vẫn nhỏ vì hình vuông đối xứng bốn lần. Chỉ bắt được bằng kiểm nhất quán liên tag (B/09 mục 7).
●● Marker tròn: cảnh báo	σ rất nhỏ (tâm xác định bởi nhiều pixel biên) nhưng chịu bias tâm ellipse — thiên lệch hệ thống phụ thuộc tư thế (A/02 mục 6). Chính xác quanh một giá trị lệch không bằng kém chính xác quanh giá trị đúng.
●● Detector học sâu	Bền hơn với mờ, che khuất, chiếu sáng khắc nghiệt. Độ chính xác góc <i>chưa chắc</i> hơn — dùng giả định bền hơn thì chính xác hơn. Lợi ích ít rõ với docking, nơi điều kiện kiểm soát được.
●●● Đòn bẩy của chương này	Thấp nhất trong Phần B. Đổi detector thay đổi σ 2–3 lần; đổi bố trí constellation thay đổi sai số góc hàng chục lần. Thứ tự đầu tư đúng: B/13, B/09, B/12, rồi mới tới đây.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
apriltag (AprilRobotics)	Bản C gốc; đọc phần khớp đường có trọng số gradient để thấy mục 2 thành code	Một trong ba kho đáng đọc mã chứ không chỉ dùng (D/17)
OpenCV objdetect	ArUco và ChArUco; ArucoDetector với tham số cornerRefinementMethod	Nhớ bật tinh chỉnh góc — tắt mặc định ở một số phiên bản
Sinh từ điển ArUco [5]	Tạo từ điển nhỏ với khoảng cách Hamming lớn, đúng thứ cần cho docking	Đường tắt tốt hơn việc dùng từ điển mặc định quá lớn
apriltag-generation	Tự tạo họ tag với số ô và khoảng cách tùy chọn	Dùng khi muốn thực hiện đánh đổi ở mục 5C
So sánh thực nghiệm các họ tag [12]	Nguồn duy nhất trong cây có số đo trực tiếp so sánh giữa các họ	Con số gắn với điều kiện đo của họ; đọc điều kiện trước khi dùng số

MINI PROJECT · Đo σ thật của hệ bạn, và so ba cấu hình detector · 1–2 ngày

- Mục tiêu** thay con số “ $\sigma \approx 0,2$ px” mượn từ kinh nghiệm bằng con số đo được của chính hệ bạn — đầu vào bắt buộc cho mọi phép tính ở C/15.
- Dữ liệu** phần cứng thật: camera cố định trên giá, tag gắn cố định, chiếu sáng ổn định, mọi tham số tự động đã khoá.

Các bước	chụp 500 khung liên tiếp không ai chạm vào gì; với mỗi khung, chạy detector và ghi toạ độ bốn góc; tính độ lệch chuẩn của từng toạ độ qua 500 khung — đó là σ của bạn; lập với ba cấu hình: AprilTag mặc định, ArUco không tinh chỉnh góc, ArUco có cornerSubPix; rồi lập toàn bộ ở ba khoảng cách và ba góc nghiêng khác nhau.
Xong khi	bạn có một bảng σ theo cấu hình, khoảng cách và góc nghiêng. Kiểm tra chéo bắt buộc <i>hai</i> cái. Một: σ phải gần như không đổi theo khoảng cách khi tính bằng pixel — nếu nó tăng mạnh khi tag nhỏ đi thì bạn đang chạm ngưỡng kích thước tối thiểu của A/02 mục 4, và đó là một kết quả quan trọng. Hai: độ tản mát bạn đo là <i>nhiều</i> chứ không gồm thiên lệch, vì mọi khung chụp cùng một cảnh — nên con số này là cận dưới của sai số thật, và đừng nhầm nó với độ chính xác.
Nối với	C/15 nhận σ này thay cho giá trị mặc định; B/08 dùng bảng này để quyết định có cần chuyển sang ChArUco không.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Cơ chế thứ nhất: bước khớp tứ giác nhằm một vật thể khác — cửa sổ, biển báo, hoạ tiết sàn, bất cứ thứ gì gần tứ giác — rồi bước giải mã tình cờ đọc ra một chuỗi bit nằm trong từ điển. Cơ chế thứ hai: có một tag thật nhưng nhiều bit làm nó giải mã sai thành ID khác, và điều này dễ xảy ra hơn khi số lỗi bit cho phép sửa được đặt cao. Hai biện pháp ở hai tầng khác nhau: ở tầng detector, giảm số lỗi bit cho phép sửa — an toàn quan trọng hơn vài chục phân trăm phát hiện; ở tầng ứng dụng, dùng whitelist ID (bạn biết trước dock có đúng những ID nào, nên mọi ID khác bị loại ngay) cộng với kiểm tính hợp lý của pose — một tag báo ở 50 m hoặc ở góc nghiêng 89° là đáng ngờ bất kể mã có hợp lệ.
- A2. Bước khớp cạnh. AprilTag 1 lấy vị trí cạnh từ ảnh *nhi phân*, nơi thông tin bị lượng tử hoá thành một pixel và toàn bộ thông tin về độ sáng đã bị vứt bỏ. AprilTag 2 quay lại ảnh *xám gốc*, lấy các pixel dọc cạnh và khớp một đường thẳng với trọng số theo độ lớn gradient [1]. Thay đổi ấy quan trọng vì hai lẽ: một cạnh đi qua giữa hai pixel để lại dấu vết trong tỉ lệ độ sáng của hai pixel đó, nên thông tin dưới pixel tồn tại và chỉ cần không vứt nó đi; và việc dùng cả trăm pixel dọc cạnh để xác định một đường làm nhiều của từng pixel bị trung bình đi. Kết quả là định vị góc chuyển từ độ chính xác cỡ một pixel xuống cỡ phần mười pixel.
- A3. Vì hai loại giải hai việc khác nhau. ArUco giải *định danh*: nó cho biết ô nào là ô nào, kể cả khi chỉ nhìn thấy một phần bàn cờ — điều mà bàn cờ thuần không làm được vì mọi góc trông giống hệt nhau. Chessboard giải *đo*: góc của nó là một điểm yên ngựa, nơi gradient đổi dấu theo *cả* hai hướng, nên vị trí bị ràng buộc từ bốn phía; trong khi góc của ArUco là giao của hai đường thẳng, chỉ bị ràng buộc bởi hai hướng và sai số của mỗi đường lan vào giao điểm với hệ số khuếch đại phụ thuộc góc giao. Sự phân công ấy chính là cơ sở cho đề xuất target lai: AprilTag hoặc ArUco để bắt bám và định danh từ xa, vùng chessboard để đo tinh ở cự ly gần.
- A4. Cái giá là *tầm phát hiện*. Tag $n \times n$ ô dữ liệu có tổng khoảng $(n+2)$ ô trên cạnh kể cả viền, và cần chừng 3–5 px mỗi ô để giải mã tin cậy, nên $w_{\min} \approx 4(n+2)$ px. Với 16h5 (4×4) thì $w_{\min} \approx 24$ px; với 36h11 (6×6) thì $w_{\min} \approx 32$ px. Vì $w = fW/Z$, khoảng cách phát hiện tối đa tỉ lệ nghịch với $w_{\min}$, nên họ ít ô cho tầm xa hơn $32/24 = 1.33$ lần với cùng kích thước tag vật lý. Với docking, số ID cần thiết thường dưới vài chục trong khi 36h11 có hàng trăm, nên đánh đổi gần như luôn nghiêng về phía ít ô hơn — và mất mát về khoảng cách Hamming được bù lại bằng whitelist ID, một biện pháp gần như miễn phí.
- A5. Đúng vì độ lớn của đòn bẩy: chuyển từ detector này sang detector khác thay đổi σ chừng hai tới ba lần, trong khi chuyển từ một tag đơn sang constellation có baseline rộng thay đổi sai số góc hàng chục lần (A/05 mục 6) và việc phá tính đồng phẳng thì khử hẳn một nguồn lỗi thô. Nó vẫn đáng đọc vì ba lý do. Một, các bước khác nhau hỏng theo những cách khác nhau — ngưỡng hoá hỏng thì tag biến mất, khớp cạnh hỏng thì σ lớn, giải mã hỏng thì có báo sai — và ba triệu chứng ấy đòi ba cách chữa khác nhau. Hai, có ba tham số đáng chỉnh và biết pipeline là biết chỉnh cái nào. Ba, và quan trọng nhất, hiểu *cơ chế* định vị góc là cơ sở cho quyết định target lai — một quyết định thuộc loại đòn bẩy cao, dù nó được rút ra từ một chương đòn bẩy thấp.

NÓI LẠI THẬT ĐƠN GIẢN

Máy tính tìm một cái tag trong ảnh theo bốn bước, và mỗi bước hông theo một kiểu riêng.

Bước đầu là tách sáng khỏi tối, nhưng phải làm riêng cho từng vùng nhỏ, vì một góc ảnh có nắng còn góc kia có bóng râm. Nếu bước này hông thì cái tag *biến mất hẳn* — không phải đo sai, mà là không thấy gì. Chữa bằng cách lo ánh sáng, không phải lo thuật toán.

Bước hai là gom các mảng tối liền nhau rồi hỏi mảng nào giống hình tứ giác. Phần lớn bị loại luôn. Chỗ hông ở đây là nhầm: một cái cửa sổ, một viên gạch, một vết bẩn vuông vắn — tất cả đều là ứng viên.

Bước ba là bước quan trọng nhất, và là chỗ quyết định độ chính xác. Ta có một mảng pixel hình tứ giác nhưng cần biết bốn góc của nó ở đâu *chính xác tới phần mười pixel*. Cách ngây thơ là lấy bốn pixel ngoài cùng — và cách đó phí phạm, vì nó dùng bốn pixel rồi vứt bỏ hàng trăm pixel dọc bốn cạnh. Cách tốt là nhìn vào ảnh xám gốc, tìm nơi độ sáng đối nhanh nhất, kẻ một đường qua tất cả những chỗ ấy, rồi lấy nơi hai đường cắt nhau. Mỗi góc bây giờ được quyết định bởi cả trăm pixel, và đó là lý do đo được dưới pixel.

Bước bốn là đọc mã, và bước này là chuyện an toàn chứ không phải chuyện chính xác. Nấn cái tứ giác về hình vuông, chia ô, đọc từng ô sáng hay tối, rồi tra xem chuỗi ấy có nằm trong danh sách hợp lệ không. Cái danh sách ấy được thiết kế rất kỹ để hai mã bất kỳ khác nhau nhiều bit — nhờ vậy một vết bẩn ngẫu nhiên hầu như không bao giờ trùng với một mã thật. Điều này quan trọng hơn về ngoài: nếu robot *không thấy* tag thì nó chỉ đứng chờ, nhưng nếu nó *thấy nhầm* thì nó lái đi chỗ khác với đầy đủ tự tin.

Còn một chuyện đáng nói và nó hơi bất ngờ: chương này là chương *ít quan trọng nhất* trong phần các phương pháp. Đối từ detector này sang detector kia làm độ chính xác tốt lên chừng hai ba lần. Đối cách xếp mấy tấm tag trên dock làm nó tốt lên hàng chục lần. Người ta dành hàng tuần để so sánh detector và dành một buổi chiều để quyết định chỗ dán tag, trong khi đáng ra nên ngược lại. Và một điểm riêng đáng nhớ: cái loại tag vuông phổ biến thì giới việc *nhận ra* — nó đọc được từ xa, chịu được bẩn, hầu như không bao giờ nhận nhầm. Nhưng nó không phải loại đo góc chính xác nhất. Cái bàn cờ vua đen trắng thì đo chính xác hơn, chỉ có điều nó không tự nói cho bạn biết bạn đang nhìn ô nào. Cách khôn nhất là dùng cả hai: tag vuông để nhận ra và bám theo từ xa, bàn cờ để đo cho chuẩn lúc đã tới gần.

Chỗ không rút gọn được: chuyện detector nào chính xác hơn detector nào, bao nhiêu lần, trong điều kiện nào — cái đó không có câu trả lời chung, vì nó phụ thuộc ống kính, ánh sáng, khoảng cách. Cách duy nhất đáng tin là đo trên chính hệ của mình, và đo thì không khó: gắn cố định mọi thứ, chụp năm trăm tám, xem con số nhảy bao nhiêu.

Một câu: Bốn bước, và chỉ bước thứ ba quyết định độ chính xác — còn cả chương này thì quyết định ít hơn nhiều so với chỗ bạn dán mấy tấm tag.

1.8 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *góc chessboard định vị chính xác hơn góc từ giao hai đường thẳng* — bị bác bỏ nếu phép đo σ ở mini project cho thấy hai loại cho độ tản mát tương đương trên cùng một hệ. Tôi dẫn mệnh đề bằng lập luận về số hướng ràng buộc (cửa a) và nó phù hợp với thực tế rằng cộng đồng hiệu chuẩn camera dùng chessboard chứ không dùng tag vuông; nhưng tôi *chưa* có số đo so sánh được (cửa b và c chưa qua). Đây là mệnh đề nền cho đề xuất target lai nên nó đáng kiểm soát.

Mệnh đề thứ hai — *chương này có đòn bẩy thấp nhất trong Phần B* — bị bác bỏ nếu trong một hệ thực, việc đổi detector cải thiện sai số cuối cùng nhiều hơn việc cải thiện constellation. Điều đó có thể xảy ra trong một trường hợp cụ thể: nếu constellation đã rất tốt và ngân sách đang bị chi phối bởi σ . Nói cách khác, mệnh đề của tôi là một phát biểu về *trường hợp điển hình*, không phải một định luật.

Mệnh đề thứ ba — *họ tag ít ô cho tầm phát hiện xa hơn theo tỉ lệ $w_{\min}$* — là phép tính đơn giản và nó chắc chắn về mặt số học; chỗ có thể sai là con số 3–5 px mỗi ô, vốn là bậc độ lớn kinh nghiệm. Nếu ngưỡng thật khác đáng kể thì tỉ số 1,33 đổi theo, nhưng chiều của kết luận không đổi.

Cuối cùng, tôi đã cố ý *không* phát biểu mệnh đề “detector X tốt hơn detector Y”, vì tôi không có dữ liệu so sánh được. Nếu bạn cần xếp hạng ấy thì nó phải đo trên cấu hình của bạn, và mini project cung cấp khung.

1.9 Kết nối

Chương này là cửa vào của Phần B và nó nối ở ba hướng.

Nối lên trên. A/01-mo-hình-camera-va-distortion mục 4 là chương phải đọc trước: nó giải thích vì sao méo chưa khử phá hoại đúng bước khớp cạnh mà mục 2 ở đây mô tả, và hai chương cùng nhau tạo thành một lập luận hoàn chỉnh về việc khử méo là bắt buộc. A/02-homography-va-hình-hoc-phang cấp bước rectify, và mục 5 ở đó (về tỉ số kép) giải thích vì sao mọi họ tag hiện đại đều mã hoá bằng lưới ô rời rạc chứ không bằng khoảng cách.

Nối ngang trong Phần B. B/08-corner-refinement là chương tiếp nối trực tiếp: chương này mô tả cơ chế định vị góc mặc định của từng detector, chương kia bàn cách cải thiện nó và đưa ra con số σ cho từng phương pháp. Hai chương nên đọc liền nhau. B/09-multi-marker-constellation là chương mà mục 1 chỉ tới khi nói về thứ tự đầu tư: nó có đòn bẩy cao hơn nhiều, và nó dùng ngưỡng kích thước tag mà chương này bàn ở mục 5C để thiết kế nested multi-scale. B/14-dieu-kien-quan-sat-va-anh-sang là chương chứa chế độ hồng của bước ngưỡng hoá — một chế độ hồng mà không thuật toán nào chứa được.

Nối xuống Phần C và D. C/15-ngan-sach-sai-so nhận σ từ mini project của chương này làm một trong bốn đầu vào của mọi phép tính. D/17-thu-vien-va-detector là chương công cụ tương ứng: nó nói API cụ thể, tên tham số, và chỗ nào trong mã nguồn hiện thực hoá từng bước mô tả ở đây.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Ba bước của pipeline hồng theo ba kiểu khác nhau. Nêu triệu chứng của từng kiểu, và nêu cách chữa — chỉ ra cách nào *không* nằm ở tầng thuật toán.
2. Vì sao khớp đường trên ảnh xám với trọng số gradient chính xác hơn nhiều so với lấy biên của ảnh nhị phân? Nêu hai cơ chế độc lập.
3. ChArUco dùng hai loại đặc trưng cho hai việc. Nêu việc nào cho loại nào, và giải thích bằng cấu trúc hình học vì sao mỗi loại giỏi việc của nó.
4. Với tag 5×5 ô dữ liệu và ngưỡng 4 px mỗi ô, kích thước ảnh tối thiểu là bao nhiêu? Nếu $f = 600$ px và $W = 80$ mm, tầm phát hiện tối đa là bao xa?
5. Nêu chế độ hồng thâm lặng nhất của chương này, giải thích vì sao sai số tái chiếu không bắt được nó, và nêu cơ chế ở chương khác bắt được.
6. Vì sao chương này có đòn bẩy thấp nhất trong Phần B, và trong trường hợp cụ thể nào thì điều đó không còn đúng?
7. Bạn đo σ bằng cách chụp 500 khung của một cảnh tĩnh. Con số thu được là cận trên hay cận dưới của sai số thật, và thành phần nào nó không đo được?

Nguồn

- [1] J. Wang and E. Olson, “AprilTag 2: Efficient and Robust Fiducial Detection,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin A; quad fitting bằng weighted line fit trên gradient, 2016.
- [2] E. Olson, “AprilTag: A Robust and Flexible Visual Fiducial System,” in *IEEE International Conference on Robotics and Automation (ICRA)*, mức tin A; bài gốc, lexicode và quy trình phát hiện, 2011.
- [3] M. Krogus, A. Haggemiller, and E. Olson, “Flexible Layouts for Fiducial Tags,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, mức tin B; April-Tag 3, các family không vuông, 2019.
- [4] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Automatic Generation and Detection of Highly Reliable Fiducial Markers under Occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014, mức tin A; ArUco, sinh từ điển bằng tối ưu khoảng cách Hamming.
- [5] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, et al., “Generation of Fiducial Marker Dictionaries Using Mixed Integer Linear Programming,” *Pattern Recognition*, vol. 51, pp. 481–491, 2016, mức tin B.
- [6] H. Ha, M. Perdoch, H. Alismail, et al., “Deltile Grids for Geometric Camera Calibration,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A; lưới tam giác và tinh chỉnh saddle point bậc hai, chính xác hơn checkerboard, 2017.
- [7] T. Krajník, M. Nitsche, J. Faigl, et al., “A Practical Multirobot Localization System,” *Journal of Intelligent & Robotic Systems*, vol. 76, pp. 539–562, 2014, mức tin B; WhyCon, marker tròn đồng tâm.
- [8] L. Calvet, P. Gurdjos, C. Griwodz, et al., “Detection and Accurate Localization of Circular Fiducials under Highly Challenging Conditions,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; CCTag, vòng tròn đồng tâm định vị chính xác, 2016.
- [9] B. Benligray, C. Topal, and C. Akinlar, “STag: A Stable Fiducial Marker System,” in *Image and Vision Computing*, mức tin B; dùng đường tròn ngoại tiếp để ổn định pose, vol. 89, 2019, pp. 158–169.
- [10] Z. Zhang, Y. Hu, G. Yu, et al., “DeepTag: A General Framework for Fiducial Marker Design and Detection,” in *IEEE Transactions on Pattern Analysis and Machine Intelligence*, mức tin B, 2023.
- [11] R. Berral-Soler, F. J. Madrid-Cuevas, R. Muñoz-Salinas, et al., “DeepArUco: Marker Detection and Classification in Challenging Lighting Conditions,” *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, mức tin C; tiền ấn/-workshop, dùng làm gợi ý.
- [12] M. Kalaitzakis, B. Cain, S. Carroll, et al., “Fiducial Markers for Pose Estimation: Overview, Applications and Experimental Comparison of the ARTag, AprilTag, ArUco and STag Markers,” in *Journal of Intelligent & Robotic Systems*, mức tin B; so sánh thực nghiệm nhiều họ tag — nguồn duy nhất trong cây có số đo trực tiếp so sánh giữa các họ, vol. 101, 2021.